

计算机程序设计基础(2)

--- C++程序设计(11)

孙甲松

sunjiasong@tsinghua.edu.cn

电子工程系 信息认知与智能系统研究所

罗姆楼6-104

电话: 62796193/13901216180

2020. 4.

9. C++标准模板库(STL)

9.1 命名空间

9.2 泛型程序设计

9.3 容器

9.4 迭代器

9.5 算法

9.6 函数对象

9.7 应用举例

9.1 命名空间(namespace)

- 命名空间将不同的标识符（对象或函数）定义在一个命名作用域内，主要目的是为了解决大型软件开发中的命名冲突。

- 例如，若声明了一个叫NS的命名空间：

```
namespace NS {  
    class File;  
    void fun();  
}
```

则引用的方式是：

```
NS::File obj; // 定义对象obj  
NS::fun();    // 调用函数fun
```

- 通常，没有命名空间的标识符都处于无名的命名空间中，也就是全局作用域。
- 命名空间可以嵌套定义，即一个命名空间中可以定义更多（子）命名空间。

9.1 命名空间(namespace)

■ 命名空间的定义方法

命名空间定义的一般格式如下：

```
namespace 空间名
{
    变量(可以带有初始化);
    常量;
    函数(可以是定义或声明);
    结构体;
    类;
    模板;
    命名空间(命名空间中嵌套定义命名空间)
}
```

全局作用域

标准命名空间

命名空间1

命名空间2

.....

命名空间n

9.1 命名空间(namespace)

- 命名空间可以起别名：namespace N2 = NS;
- 可以用using来指定默认命名空间。
- 比如，若有以下声明：

using NS::File;

则在有效作用域内，使用File定义对象不用加NS::，默认File是NS空间的：

File obj;

- using namespace NS;

就是指定NS命名空间，则随后的语句可以直接引用NS中的符号和函数，不用再加NS::限定名

File obj;

Fun();

9.1 命名空间(namespace)

- `using namespace std;` 就是指定了std标准命名空间，其中的所有标识符都可以直接引用。
- 使用标准命名空间，所有引用的头文件都不再使用扩展名 “.h”，比如：

```
# include <iostream> // 会出现ios_base与ios的区别  
# include <cstring> // C的.h文件去.h后前面加c  
using namespace std;
```

- 若两个命名空间中有同名的标识符，使用时最好加上对应命名空间的限定名，比如：

NS::a std::a

- 否则可能会导致因为打开不同的命名空间而产生不同的结果（有的结果肯定不对）。

■ 命名空间的定义和使用举例：

```
#include <iostream>
using namespace std;
int const Num = 1; //全局作用域
namespace Name1
{
    int Num = 8; // 命名空间作用域
    int Add(int Num2) // Num2 函数作用域
    {
        Num2 = ::Num + Num + Num2; //::表示无名空间，全局作用域
        return Num2;
    }
}
namespace Name2
{
    int Num = 9;
    int Add(int Num)
    {
        Num = ::Num + Name2::Num + Num;
        return Num;
    }
}
```



```

void main(void)
{
    cout << Name1::Add(4) + ::Num << endl;
    cout << Name2::Add(5) + Num << endl;
    namespace N2 = Name1;
    cout << N2::Add(6) + N2::Num << endl;
    using namespace Name2;
    cout << Add(7) + Name2::Num << endl;
}

```

程序运行结果为:

14

16

23

26

请按任意键继续...

说明: Name1命名空间中Add函数中:

Num2 = ::Num + Num + Num2;

是将全局变量Num与当前命名空间中的Num以及函数形参Num2求和。

而Name2命名空间中Add函数中:

Num = ::Num + Name2::Num + Num;

是将全局变量Num与Name2命名空间中的Num以及函数形参Num求和。

如果把主程序最后一行改为:

```
cout << Add(7) + Num << endl;
```

编译结果:

test.cpp(29) : error C2872: “Num”: 不明确的符号

1> 可能是 “test.cpp(3) : const int Num”

1> 或 “test.cpp(15) : int Name2::Num”

原因是:

```
cout << Add(7) + Num << endl;
```

语句前面的一行:

```
using namespace Name2;
```

指定Name2为默认命名空间, Add(7) 前不必再加Name2::限定命名空间。但因为Num是全局变量, 同时也是Name2命名空间中的变量, 编译器将无法确定是哪一个Num, 因此必须写为: Name2::Num 指明是Name2命名空间中的Num变量, 写为 ::Num 也可以避免出现编译错误。

9.1.1 命名空间与作用域

- 命名空间的引入，进一步细化了函数和对象的作用域。
- 作用域由小到大依次为：复合语句作用域、函数作用域、文件作用域、类作用域、命名空间作用域、全局作用域。

- 复合语句作用域

在任意复合语句`{ }`中定义的局部变量，其作用域仅限在局部小范围内。

- 函数作用域

在函数体内`{ }`中定义的局部变量，其作用域仅限在本函数内。

9.1.1 命名空间与作用域

- 文件作用域

全局变量或函数前加上static，使得其作用域仅限于源程序的本文件中。

- 类作用域

类中所封装的成员变量或成员函数，其作用域仅限于本类对象中。

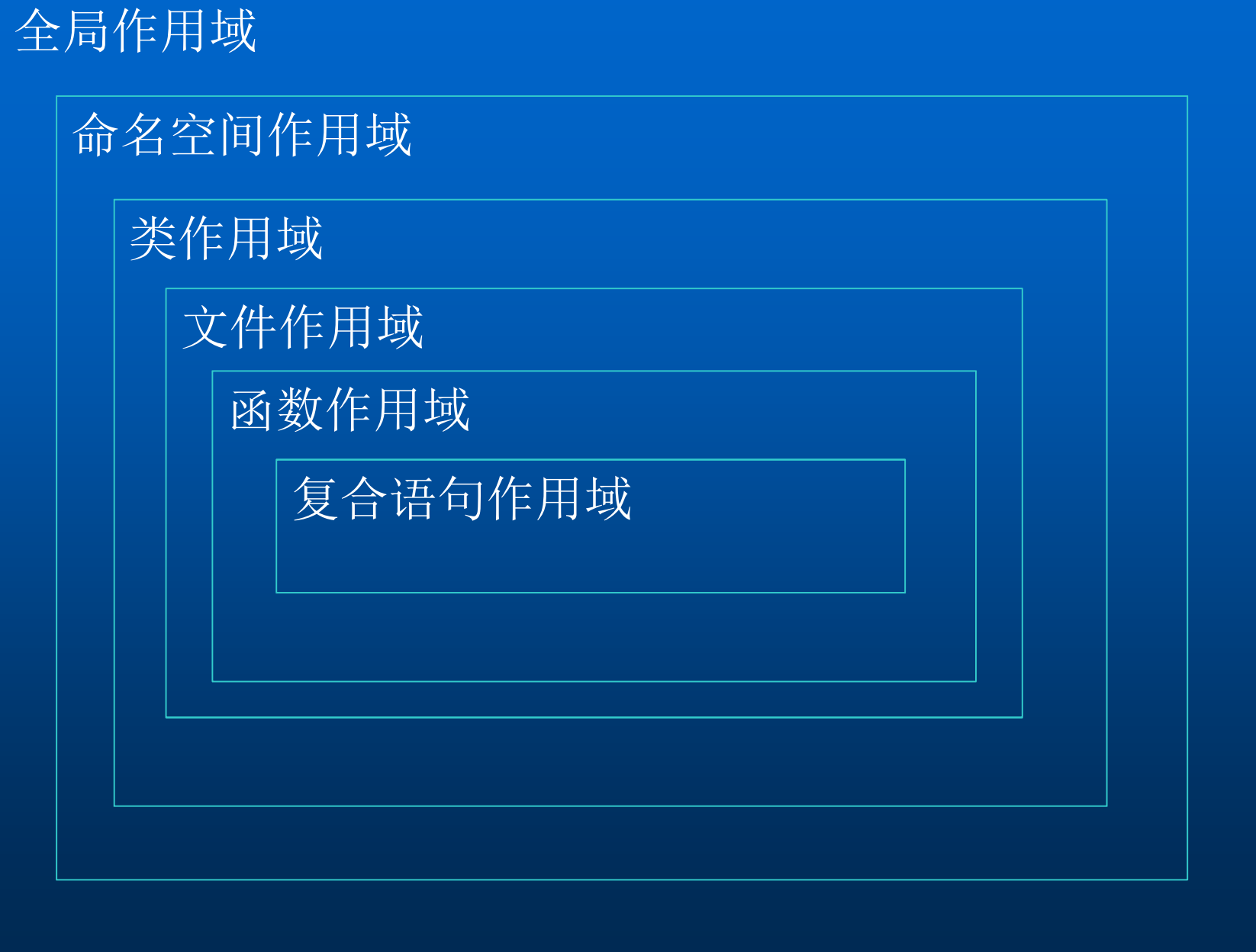
- 命名空间作用域

将不同的对象（变量）或函数定义在一个命名作用域内，其作用域仅限于本命名空间。

- 全局作用域

将对象（变量）或函数定义为全局变量，其作用域为全程序全局可见。

全局作用域



命名空间作用域

类作用域

文件作用域

函数作用域

复合语句作用域

9.2 泛型程序设计

- 泛型程序设计是指将程序写得尽可能通用，便于程序复用，便于开展大规模工业化软件开发。
- 将算法从特定的数据结构和类型中抽象出来，使之成为通用的算法(模板)。
- 模板是泛型程序设计的基础。
- STL (Standard Template Library, 标准模板库) 是泛型程序设计的范例。
- STL最初是由HP公司的Alexander Stepanov和Meng Lee开发的，后来由包括这2人在内的4人组进行了完善优化。1994年成为了C++标准的一部分。不同编译器的STL的实现有不同的实现版本，估计一直在优化改进，但用户接口（调用方式）都遵守共同的标准。

9.2 泛型程序设计

- STL最关键的4个基本组件是：
 - ① 容器(container)
 - ② 迭代器(iterator)
 - ③ 函数对象(function object)
 - ④ 算法(algorithms)

使用举例：从键盘上读入若干个数放入容器中，求反后逐个输出。

```
#include <iostream>
#include <vector>      // 向量容器头文件
#include <iterator>    // 迭代器头文件
#include <algorithm>   // 算法头文件
#include <functional> // 函数对象头文件
using namespace std ;
void main( )
{  const int N=10;
   vector<int> s(N); // 定义一个大小为N的向量容器
   int n;
   for (n=0; n<N; n++)
       cin >> s[n];
   transform(s.begin( ), s.end( ),
             ostream_iterator<int>(cout, " "), negate<int>( ) );
   cout << endl;
}
```


运行结果:

1 2 3 4 5 6 -7 -8 -9 -10

-1 -2 -3 -4 -5 -6 7 8 9 10

请按任意键继续...

此例用到了STL的全部4个基本组件:

容器 `vector`

迭代器 `ostream_iterator`

函数对象 `negate`

算法 `transform`

以及对容器对象内容的遍历方法:

`s.begin()`, `s.end()`

9.3 容器

- 容器类是包含一组元素的对象。
- 现在的STL容器类模板库包含7种基本容器：
向量（vector）、双端队列（deque）、列表（list）、集合（set）、多重集合（multiset）、映射（map）、多重映射（multimap）
- 分为：① 顺序容器(sequence container)
将一组具有相同类型的元素以严格的线性形式组织起来。向量、双端队列、列表属于这一类。
② 关联容器(associative container)
具有根据一组索引快速存取元素的能力。集合、映射、多重集合和多重映射属于这一类。
- 也可称为：同类容器类、异类容器类

9.3 容器

- 容器运算符op有6个：

==, !=, >, >=, <, <=, =

若s1、s2是两个容器对象，那么可以通过：

s1 op s2

对两个容器之间的元素按字典序进行比较。

- 容器的方法（函数）：

访问方法：size(), swap(), empty(), clear()

s1.swap(s2) 将s1和s2容器的内容交换。

s1.clear() 将s1容器的内容清空。

s1.empty() 判断s1容器是否为空。

s1.size() 求s1容器的大小（元素个数）。

9.3 容器

- 迭代方法: `begin()`, `end()`, `rbegin()`, `rend()`
- 分为正向迭代器和逆向迭代器。
- 正向迭代器通过`begin()`和`end()`函数返回值提供给算法, 迭代类型可以用以下方式表示:
- `S::iterator` 表示与S相关的普通迭代器类型, 迭代器指向元素的类型为T。
- `S::const_iterator` 表示与S相关的常迭代器类型, 迭代器指向元素的类型为`const T`, 只能通过迭代器读取元素, 不能通过迭代器改写元素, 此时容器对象s1为常对象。
- `s1.begin()` 返回指向s1第一个元素的迭代器。
- `s1.end()` 返回指向s1最后一个元素的下一个位置的迭代器, 通过++遍历容器内元素。

9.3 容器

- 逆向迭代器要求容器是可逆容器,可逆容器所提供的迭代器是双向迭代器,可以双向遍历容器。
- 逆向迭代器通过`rbegin()`和`rend()`函数返回值提供给算法, 迭代类型可以用以下方式表示:
- `S::reverse_iterator` 表示与S相关的普通迭代器类型, 迭代器指向元素的类型为T。
- `S::const_reverse_iterator` 表示与S相关的常迭代器类型, 迭代器指向元素的类型为`const T`, 只能通过迭代器读取元素, 不能通过迭代器改写元素, 此时容器对象s1为常对象。
- `s1.rbegin()` 返回指向s1最后一个元素的迭代器。
- `s1.rend()` 返回指向s1第一个元素的前一个位置的迭代器,通过++(实际映射为--)遍历容器内元素。

可以把一个容器的内容逆向输出到标准输出中：

```
#include <iostream>
#include <vector>      // 向量容器头文件
#include <iterator>    // 迭代器头文件
#include <algorithm>   // 算法头文件
#include <functional> // 函数对象头文件
using namespace std ;
void main( )
{  const int N=10;
   vector <int>  s(N); // 定义一个大小为N的向量容器
   int n;
   for (n=0; n<N; n++)
       cin >> s[n];
   transform(s.rbegin( ), s.rend( ),
             ostream_iterator<int>(cout, " "), negate<int>( ) );
   cout << endl;
}
```

运行结果:

1 2 3 4 5 6 -7 -8 -9 -10

10 9 8 7 -6 -5 -4 -3 -2 -1

请按任意键继续...

- 另外, `vector`和`deque`除了是顺序容器外, 还是可随机访问容器, 可以通过`s[n]`对容器的第`n`个元素进行随机访问。这里`s[n]`等价于`s.begin()[n]`。
- 顺序容器还有构造函数、赋值函数`assign`、元素插入函数`insert`、元素删除函数`delete`、改变容器大小函数`resize`、访问容器首尾元素函数`front`和`back`、在容器头部插入删除元素函数`push_front`和`pop_front`、在容器尾部插入删除元素函数`push_back`和`pop_back`等。有兴趣可以自己看一下, 时间所限不再细讲。

9.3 容器

■ 适配器（adapter）

是一种接口类，为已有的类提供新的接口，目的是简化、约束、让其安全、隐藏或者改变被修改类提供的服务集合。

一共有三种适配器：容器适配器（扩展7种基本容器）、迭代器适配器、函数对象适配器。

■ 顺序容器的适配器

由于顺序容器支持在任意位置的插入和删除操作，允许随机访问容器中的任意元素，有时认为这过于灵活，需要加以限定，希望按照指定的顺序来访问容器中的元素。比如，栈操作的后进先出和队列操作的先进先出。由此STL提供了容器适配器栈(stack)和队列(queue)，这是2个类模板。


```

#include <iostream>
#include <stack>    //包含栈容器头文件
using namespace std ;
void main(void)
{ stack <int> s; // 用来存放质数的向量,初始有10个元素
  int n,i;
  cout << "Enter a value >= 1 : ";    cin >> n;
  for(i = 0; i < n; i++)
    s.push(i);
  cout << "Pop from the stack: ";
  while(!s.empty( ))
  {   cout << s.top( )<< " ";
      s.pop( );
  }
}

```

运行结果:
Enter a value >= 1 : 10
Pop from the stack: 9 8 7 6 5 4 3 2 1 0请按任意键继续...

9.4 迭代器

- 迭代器是面向对象版本的泛化的指针，它们提供了访问容器、序列中每个元素的方法。
- 迭代器是算法和容器之间的“中间人”。
- 迭代器可以调用begin()、end()、next()、previous()等成员函数顺序遍历容器。
- 可通过++、*、->等运算符访问其中的元素。
- 迭代器分为：
 - ① 输入迭代器
 - ② 输出迭代器
 - ③ 前向迭代器
 - ④ 双向迭代器
 - ⑤ 随机访问迭代器

9.4 迭代器

- 输入流迭代器

从一个输入流中连续地输入某种类型的数据，它是一个类模板：

```
template <class T> istream_iterator <T>;
```

- 输出流迭代器

向一个输出流中连续地输出某种类型的数据，它也是一个类模板：

```
template <class T> ostream_iterator <T>;
```

应用举例：从标准输入读入若干个数，分别求平方后输出。

```
#include <iostream>
#include <iterator>    // 迭代器头文件
#include <algorithm>  // 算法头文件
using namespace std ;
inline double square(double x)
{ return x*x; }
void main( )
{ transform(istream_iterator<double>(cin),
            istream_iterator<double>( ),
            ostream_iterator<int>(cout, " "), square );
  cout << endl;
}
```

运行结果:

1 2 3 4 5 6

1 4 9 16 25 36

7 8 9 10

49 64 81 100 ^Z

注意: 通过Ctrl-Z键结束键盘输入

请按任意键继续...

9.5 算法

- C++标准模板库中包含70多个算法。
算法本身就是一种函数模板。
常用算法：查找、排序、消除、计数、比较、变换、置换和容器管理等统统包括。
- 这些算法特点是非常统一、标准，可以应用于不同的数据类型和对象。

- STL几乎封装了所有的数据结构中的算法，从链表到队列，从向量到堆栈，从Hash到二叉树，从搜索到排序，从增加到删除.....可以说，如果你理解了STL，你会发现你已不用拘泥于算法本身，从而站在巨人的肩膀上去考虑更高级的应用。
- 排序是最广泛的算法之一，下面简要介绍STL中不同排序算法的用法和区别。

- 所有的sort算法的参数都需要输入一个范围, [begin, end]。这里使用的迭代器(iterator)都需是随机迭代器(Random Access Iterator), 也就是说可以随机访问的迭代器, 如: it+n什么的 (partition 和stable_partition 除外)。
- 如果你需要自己定义比较函数, 你可以把你定义好的仿函数(functor)作为参数传入。每种算法都支持传入比较函数。
- 以下是所有STL sort算法函数的名字列表:

<u>函数名</u>	<u>功能描述</u>
sort	对给定区间所有元素进行排序
stable_sort	对给定区间所有元素进行稳定排序
partial_sort	对给定区间所有元素部分排序
partial_sort_copy	对给定区间复制并排序
nth_element	找出给定区间的某个位置对应的元素
is_sorted	判断一个区间是否已经排好序
partition	使得符合某个条件的元素放在前面
stable_partition	相对稳定的使得符合某个条件的元素放在前面

- `nth_element` 函数是用来找出排序后第n个元素的值。例如：找出包含7个元素的数组中排在中间那个数的值。此时，我可能不关心前面，也不关心后面，我只关心排在第4位的元素值是多少。

sort 中的比较函数

- 当你需要按照某种特定方式进行排序时，你需要给sort指定比较函数，否则程序会自动提供给你一个比较函数。

```
vector < int > vect;
```

```
//...
```

```
sort(vect.begin(), vect.end());
```

```
//此时相当于调用
```

```
sort(vect.begin(), vect.end(), less<int>());
```

- 上述例子中系统自己为sort提供了less仿函数。在STL中还提供了其他仿函数，以下是仿函数列表：

名称	功能描述
equal_to	相等
not_equal_to	不相等
less	小于
greater	大于
less_equal	小于等于
greater_equal	大于等于

- 需要注意的是，这些函数不是都能适用于你的sort算法，如何选择，决定于你的应用。另外，不能直接写入仿函数的名字，而是要写其重载的()函数：
 - less<int>()
 - greater<int>()
- 当你的容器中元素是一些标准类型（int float char)或者string时，你可以直接使用这些函数模板。
- 但如果你是自己定义的类型或者你需要按照其他方式排序，你可以有两种方法来达到效果：
 - ✓ 自己写比较函数
 - ✓ 重载类型的'<'操作赋

```

#include <iostream>
#include <algorithm>
#include <functional>
#include <vector>
using namespace std;
class myclass
{ public:
    int first;
    int second;
    myclass(int a, int b) : first(a), second(b){ }
    myclass(const myclass &p)
        : first(p.first), second(p.second){ }
    // 这里必须加const, 否则会出现编译错误:
    // error C2558: 没有可用的复制构造函数或复制构造函数声明为“explicit”
    bool operator < (const myclass &m) const
    {   return first < m.first;   }
};
bool less_second(const myclass &m1, const myclass &m2)
{   return m1.second < m2.second; }

```

```

void main( )
{
    vector< myclass > vect;
    unsigned int i;
    for(i = 0 ; i < 10 ; i++)
    {
        myclass my(10-i, i*3);
        vect.push_back(my);
    }
    for(i = 0 ; i < vect.size( ); i++)
        cout<<"("<<vect[i].first<<","<<vect[i].second<<")\n";
    sort(vect.begin( ), vect.end( )); // less
    cout<<"after sorted by first:"<<endl;
    for(i = 0 ; i < vect.size( ); i++)
        cout<<"("<<vect[i].first<<","<<vect[i].second<<")\n";
    cout<<"after sorted by second:"<<endl;
    sort(vect.begin( ), vect.end( ), less_second);
    for(i = 0 ; i < vect.size( ); i++)
        cout<<"("<<vect[i].first<<","<<vect[i].second<<")\n";
}

```

C:\Windows\system32\cmd.exe

```
<10,0>  
<9,3>  
<8,6>  
<7,9>  
<6,12>  
<5,15>  
<4,18>  
<3,21>  
<2,24>  
<1,27>
```

after sorted by first:

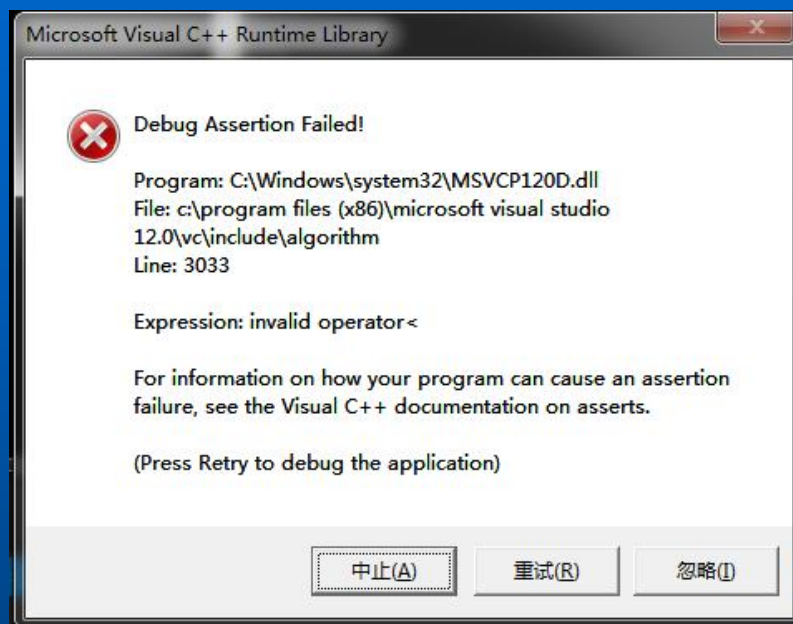
```
<1,27>  
<2,24>  
<3,21>  
<4,18>  
<5,15>  
<6,12>  
<7,9>  
<8,6>  
<9,3>  
<10,0>
```

after sorted by second:

```
<10,0>  
<9,3>  
<8,6>  
<7,9>  
<6,12>  
<5,15>  
<4,18>  
<3,21>  
<2,24>  
<1,27>
```

请按任意键继续. . .

- 在c++编程中使用sort函数，自定义一个数据结构并进行排序时，新手经常会碰到这种断言错误：



- 这是为什么呢？原因是什么？如何解决？
- 看下面一个例子：

有n个学生实体，每个学生实体包括：姓名string name; 成绩int score; 按成绩由高到低排名，成绩相同时，按姓名的字典序排序。试用STL中的sort实现。

```
#include<iostream>
#include <iterator>
#include <vector>
#include <algorithm>
#include <functional>
using namespace std;
class student
{
    public:
        char name[20];
        int score;
        student(char *p, int n)
        { strcpy_s(name, p);
          score=n;
        }
};
```

```

bool comp(const student &a,const student &b)
{
    if(a.score>b.score) return 1;
    if(a.score<b.score) return 0;
    if(strcmp(a.name,b.name)) return 1;
    return 0;
}
void main( )
{
    int i=0;
    vector <student> a;
    student stu[5]={student("yrj",99),student("abc",98),
                    student("abb",98),student("aba",98),
                    student("sgh",97)};

    for(i=0;i<5;i++)
        a.push_back(stu[i]);
    sort(a.begin(),a.end(),comp);
    for(i=0;i<5;i++)
        cout<<a[i].name<<"  "<<a[i].score<<endl;
} // 但该程序运行会出现断言错误问题。

```


由于strcmp(a,b)返回值有1、0、-1三种情况，分别代表a的字典序：大于、等于、小于b的字典序。

```
if(strcmp(a.name,b.name)) return 1;
```

将导致不论a的名字比b的名字大还是小，都将return 1；这也就意味着，你定义的比较函数，将两个参数交换时可以得到相同的结果。要注意的是，用于排序的比较函数，应当定义严格的偏序关系，不可模棱两可，即交换参数位置时应当得到完全相反的结果。

只需将 `if(strcmp(a.name,b.name))`

改成 `if(strcmp(a.name,b.name)>0)`

即可解决断言错误问题。

```
bool comp(const student &a,const student &b)
{
    if(a.score>b.score) return 1;
    if(a.score<b.score) return 0;
    if(strcmp(a.name,b.name)>0) return 1;
    return 0;
}
```

9.6 函数对象

- 一个行为类似函数的对象，它可以不需要参数，也可以带有若干参数，其功能是获取一个值，或者改变操作的状态。
- 任何普通的函数和任何重载了运算符operator()的类的对象都满足函数对象的特征。
- STL定义了一些标准的函数对象，可以分为：
 算术运算、关系运算、逻辑运算 三大类。
- 要使用这些标准函数对象，需要引用<functional>
 即： #include <functional>

9.7 使用举例

- 顺序容器——向量，动态结构，不定长数组，大小可以在程序运行过程中随时改变。
- 要使用这些标准容器，需要引用相应容器的英文名字：向量（vector）、双端队列（deque）、列表（list）、集合（set）、多重集合（multiset）、影射（map）、多重影射（multimap）
- 使用向量需要： `#include <vector>`
- 例1：前面讲过的从键盘上读入N，求2~N之间质数。

```
#include <iostream>
#include <iomanip>
#include <vector> //包含向量容器头文件
using namespace std ;
void main(void)
{ vector<int> A(10); // 用来存放质数的向量,初始有10个元素
  int n;           //质数范围的上限, 运行时输入
  int primecount = 0, i, j;
  cout << "Enter a value >= 2 as upper limit for prime numbers: ";
  cin >> n;
  A[primecount++] = 2; // 2是一个质数
  for(i = 3; i < n; i++)
  { if (primecount == A.size( ))// 若满则再增加10个元素的空间
      A.resize(primecount + 10);
    if (i % 2 == 0) //大于2的偶数不是质数,
      continue;
```

```

// 检查3,5,7,...,i/2是否i的因子
j = 3;
while (j <= i/2 && i % j != 0)
    j += 2;

if (j > i/2)    // 若上述参数均不为i的因子,则i为质数
    A[primecount++] = i;
}
for (i = 0; i < primecount; i++) //输出质数
{
    cout << setw(5) << A[i];
    if ((i+1) % 10 == 0)    //每输出10个数换行一次
        cout << endl;
}
cout << endl;
}

```

运行结果:

Enter a value ≥ 2 as upper limit for prime numbers: 1000

```
2  3  5  7 11 13 17 19 23 29
31 37 41 43 47 53 59 61 67 71
73 79 83 89 97 101 103 107 109 113
127 131 137 139 149 151 157 163 167 173
179 181 191 193 197 199 211 223 227 229
233 239 241 251 257 263 269 271 277 281
283 293 307 311 313 317 331 337 347 349
353 359 367 373 379 383 389 397 401 409
419 421 431 433 439 443 449 457 461 463
467 479 487 491 499 503 509 521 523 541
547 557 563 569 571 577 587 593 599 601
607 613 617 619 631 641 643 647 653 659
661 673 677 683 691 701 709 719 727 733
739 743 751 757 761 769 773 787 797 809
811 821 823 827 829 839 853 857 859 863
877 881 883 887 907 911 919 929 937 941
947 953 967 971 977 983 991 997
```

请按任意键继续...

例2：根据输入的n打印杨辉三角形。

```
#include <iostream>
#include <iomanip>
#include <queue>    // 包含队列容器头文件
using namespace std ;
void main(void)
{   queue<int> q;
    int s = 0, i, j, n;
    cout << "Enter a value >= 2 : ";
    cin >> n;
    q.push(1);    // 首先在队列中存入第1行元素1
    for(i = 0; i <=n; i++)
    {   cout << endl; // 对于每一行输出换行符
        cout << setw((n-i)*2) << ""; // 设置输出格式
        q.push(0); // 在队列的每一行数据中间添加一个0
```

```
for(j=1; j <= i+2; j++)  
{ int t=q.front( ); // 获取队首 (队列第一个)元素  
  q.pop( );  
  q.push(s+t);    // 保存两项之和  
  s=t;  
  if (j != i+2) cout <<setw(4)<<s; // 不打印i+2项的0  
}  
}  
cout <<endl<<endl;  
}
```

运行结果:


```
C:\Windows\system32\cmd.exe
Enter a value >= 2 : 10

      1
     1 1
    1 2 1
   1 3 3 1
  1 4 6 4 1
 1 5 10 10 5 1
1 6 15 20 15 6 1
1 7 21 35 35 21 7 1
1 8 28 56 70 56 28 8 1
1 9 36 84 126 126 84 36 9 1
1 10 45 120 210 252 210 120 45 10 1

请按任意键继续. . .
```

```
C:\Windows\system32\cmd.exe
Enter a value >= 2 : 12

      1
     1 1
    1 2 1
   1 3 3 1
  1 4 6 4 1
 1 5 10 10 5 1
1 6 15 20 15 6 1
1 7 21 35 35 21 7 1
1 8 28 56 70 56 28 8 1
1 9 36 84 126 126 84 36 9 1
1 10 45 120 210 252 210 120 45 10 1
1 11 55 165 330 462 462 330 165 55 11 1
1 12 66 220 495 792 924 792 495 220 66 12 1

请按任意键继续. . .
```

第11次作业:

见网络学堂第11次作业
2题必做

1. 试编写一个程序处理动态内存申请
`new(std::nothrow)`不成功、使用动态数组上溢出、下溢出、`delete`的指针为空等问题，使用C++的异常处理机制`throw`, `try-catch`中至少有3类以上的错误处理信息提示 (自己随便设计)。

2. 有n个学生实体，每个学生实体包括：

学号`int sno`; 姓名`string name`; 成绩`int score`;

按成绩由高到低排名，成绩相同时，按姓名的字典序排序，若出现姓名相同，则按学号从小到大排序。试用STL中的`sort`实现。